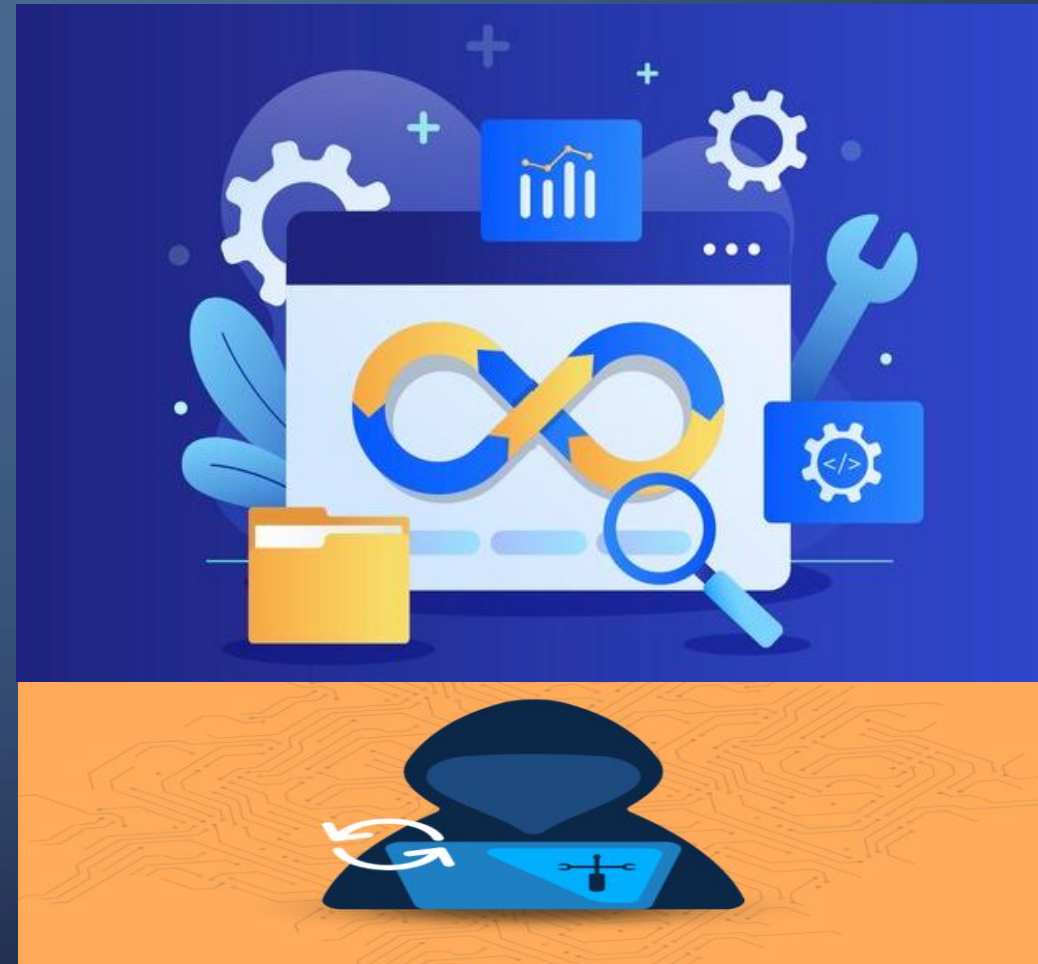


SOFTWARE SECURITY ANALYSIS AND REVERSE ENGINEERING (VULNERABILITY IDENTIFICATION, ANALYSIS, AND MITIGATION)

Hashan Kodippilige
Software Security Analysis - Project 3



INTRODUCTION

- Software security is critical in modern computing systems
- Vulnerabilities can lead to data breaches and system compromise
- Common causes include -
 - Insecure coding practices
 - Logic errors
 - Poor input validation
- Reverse engineering and analysis tools help identify hidden flaws in compiled binaries
- Understanding vulnerabilities is essential for developing secure and resilient software

GOALS

- Identify common software vulnerabilities
- Analyze vulnerabilities using static and dynamic techniques
- Perform reverse engineering on binaries
- Patch and fix software flaws
- Evaluate software protection mechanisms
- Analyze malware behavior and identify security threats

TOOLS USED

- **IDA Free** - Static & dynamic binary analysis
- **Ghidra** - Decompilation and malware analysis
- **Claude AI** - Assistance in malware interpretation
- **VirusTotal** - File reputation & hash verification
- **Linux/Kali Tools** - Hashing (SHA256), analysis utilities

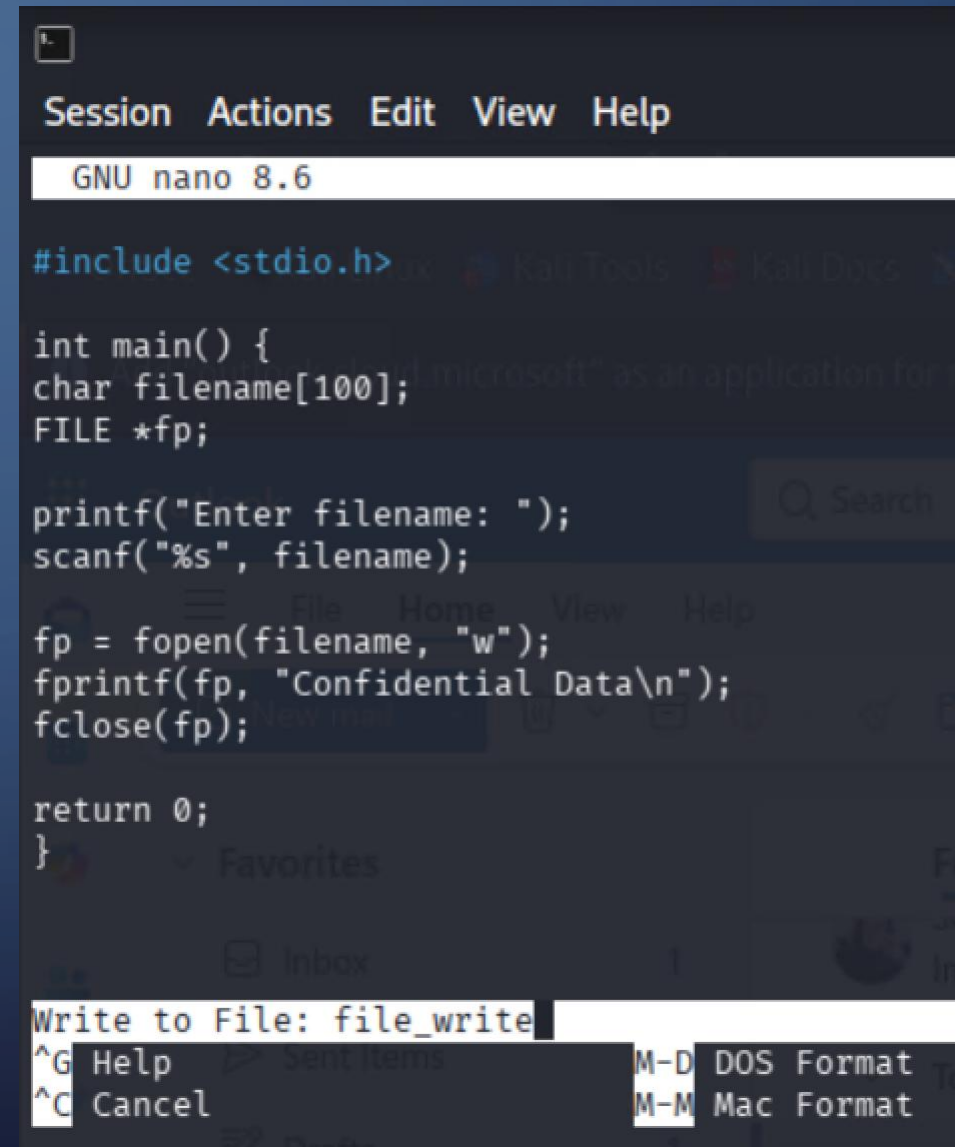
PART 1 - FILE WRITING VULNERABILITY IN C

- Program with user input is used directly in file operations
- No validation or restriction on file paths
- Can lead to -
 - Unauthorized file access
 - Overwriting critical system files
 - Data corruption

PART 1 - FILE WRITING VULNERABILITY IN C

- **Issue -**

- User controls filename
- No validation –
 - attacker can input -
 - ../../etc/passwd



```
GNU nano 8.6

#include <stdio.h>

int main() {
char filename[100];
FILE *fp;

printf("Enter filename: ");
scanf("%s", filename);

fp = fopen(filename, "w");
fprintf(fp, "Confidential Data\n");
fclose(fp);

return 0;
}
```

Write to File: file_write

^G Help ^M DOS Format
^C Cancel ^M Mac Format

COUNTERMEASURES

- Validate and sanitize user input.
- Block path traversal patterns (ex- ..)
- Restrict file access to a safe directory.
- Avoid direct use of user-provided filenames.

```
Session  Actions  Edit  View  Help
GNU nano 8.6
#include <stdio.h>
#include <string.h>

int main() {
    char filename[100];

    fgets(filename, sizeof(filename), stdin);

    // Basic validation
    if (strstr(filename, "..") != NULL) {
        printf("Invalid filename\n");
        return 1;
    }

    FILE *fp = fopen("safe_dir/output.txt", "w");
    if (fp == NULL) {
        printf("Error opening file\n");
        return 1;
    }

    fprintf(fp, "Safe Data\n");
    fclose(fp);

    return 0;
}
```

Write to File: file_write

PART 2- SOFTWARE VULNERABILITY -INJECTION FLAWS

- Untrusted input is sent to an interpreter
- Application treats input as part of a command or query
- Lack of validation allows attackers to manipulate execution
- Common Types - SQL Injection, OS Command Injection, and Cross-Site Scripting (XSS)
- Impact - Unauthorized data access, Data modification, and Full system compromise

PART 2- SQL INJECTION

- A type of injection attack targeting databases
- Attackers insert malicious SQL code into input fields
- Example Input - ' OR '1'='1'
 - Bypasses authentication
 - Returns all records from database
- Risks
 - Access to confidential data
 - Privilege escalation
 - Full administrative control

PART 2- COUNTERMEASURES

- Validate and Sanitize User Input
 - Ensure input is checked and cleaned before processing
- Enforce Least Privilege & RBAC
 - Limit database access based on roles
 - Prevent unauthorized actions

PART 3- VULNERABILITY DISCLOSURE GUIDELINES

Responsible Reporting

- Report vulnerability privately to the vendor/organization
- Provide -
 - Description
 - Impact
 - Steps to reproduce
- Avoid public disclosure before a fix is available

PART 3- VULNERABILITY DISCLOSURE GUIDELINES

Coordination (CISA CVD Model)

- Follow a coordinated vulnerability disclosure process
- Work with vendor to develop and test fixes
- Allow time for patching (responsible timeline)
- Maintain communication with stakeholders

PART 3- VULNERABILITY DISCLOSURE GUIDELINES

Public Disclosure

- Disclose vulnerability after mitigation is available
- Share Technical details and Recommended fixes

Ethical Practices

- Do not exploit the vulnerability
- Protect user data and systems
- Follow legal and professional standards

PART 4- SOFTWARE SECURITY ANALYSIS TECHNIQUES

Static Analysis

- Analysis of code without executing the program
- Examines Program structure, Logic, and functions
- Used to identify -
 - Vulnerabilities
 - Unsafe coding practices

PART 4- SOFTWARE SECURITY ANALYSIS TECHNIQUES

Advantages

- Safe (no execution required)
- Early detection of vulnerabilities
- Covers entire code

Limitations

- Cannot detect runtime behavior
- May produce false positives

PART 4- SOFTWARE SECURITY ANALYSIS TECHNIQUES

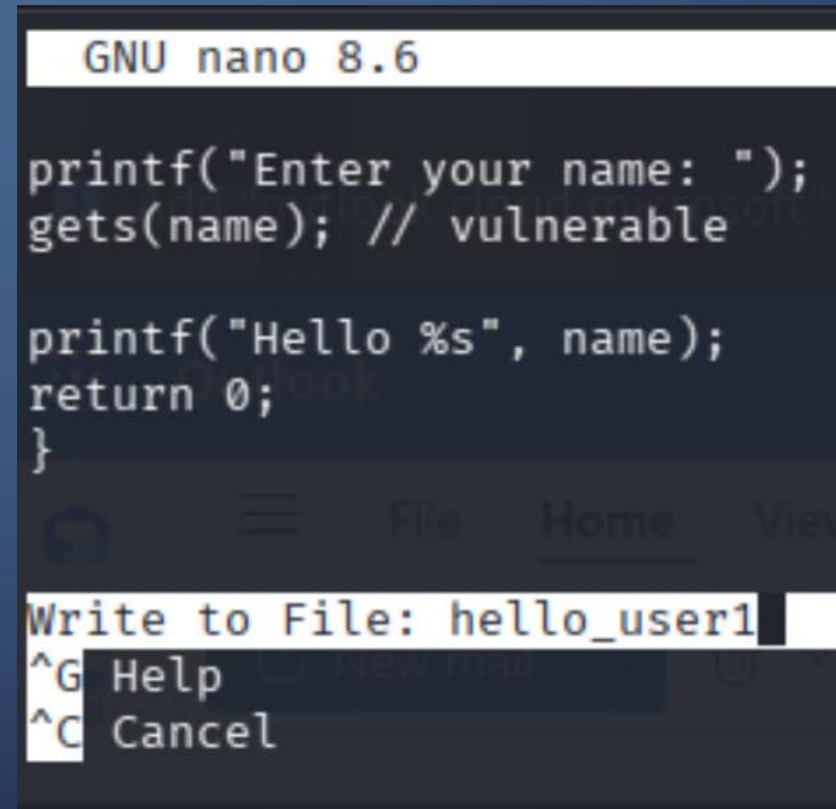
Tool - IDA Free

- Reverse engineering tool for binaries
- Provides -
 - Disassembly view
 - Pseudocode view (F5)
 - Function analysis

PART 4- SOFTWARE SECURITY ANALYSIS TECHNIQUES

Demonstration

- Analyze a simple C program
- Identify an unsafe function
- Detect a potential vulnerability



```
GNU nano 8.6

printf("Enter your name: ");
gets(name); // vulnerable

printf("Hello %s", name);
return 0;
}
```

Write to File: hello_user1

^G Help
^C Cancel

The screenshot shows a terminal window with the GNU nano 8.6 editor. The code is a simple C program that prompts the user for their name and prints it. The use of the `gets()` function is highlighted as a vulnerability. The terminal shows the program being saved to a file named `hello_user1`.

PART 5-7 DEMO

CONCLUSION

- Software vulnerabilities often originate from flawed logic, improper validation, and insecure coding practices
- Static analysis (IDA, Ghidra) enabled identification of program structure, functions, strings, and hidden behaviors
- Dynamic analysis (debugging) revealed real-time execution, conditional branching, and licensing enforcement mechanisms
- Binary patching demonstrated how program logic flaws (ex- GPA classification, license checks) can be exploited and corrected
- Licensing mechanisms (time-based trials) can be bypassed if not securely implemented

CONCLUSION

- Malware analysis exposed stealth techniques such as –
 - Registry autorun persistence
 - Hidden file creation
 - TLS callbacks executing before main()
- Hashing and VirusTotal analysis confirmed file identity and provided threat intelligence insights
- Overall, reverse engineering tools are critical for vulnerability discovery, security assessment, and understanding both legitimate and malicious software behavior

REFERENCES

- Álvarez Pérez, D., & Tiwari, R. (2025). *Ghidra software reverse-engineering for beginners* (2nd ed.). Packt Publishing.
- Fox, N. (2023, April 6). *How to use Ghidra to reverse engineer malware*. Varonis. <https://www.varonis.com/blog/how-to-use-ghidra>
- Siahaan, C. N. (2022, October 7). *Binary patching with IDA Pro (part 1)*. Medium. https://medium.com/@k_kisanak/binary-patching-with-ida-pro-part-1-c806d0f20d22
- JFrog. (n.d.). What is a software vulnerability? JFrog Learn. <https://jfrog.com/learn/devsecops/software-vulnerability/>
- Pruitt, M. (2025, May 15). Software security vulnerabilities developers need to know about. Kiuwan. <https://kiuwan.com/blog/types-software-vulnerabilities/>

THANK YOU!